

A-LEVEL COMPUTER SCIENCE PROJECT

PROJECT NAME: Classroom Availability Project (CLAVP)

By: Morgan McCabe

PROJECT ANALYSIS

Problem identification

At the school I currently attend to there seems to be a lack of communication of the classrooms schedule. Teachers with their class following them would be searching for an empty room. Other times people who believed the lass to be free would sooner or later get kicked out due to a class being booked in for that time. There was a clear issue that seemed to be commonplace in the day to day running of the school. Schools need to organise multiple individual children in the hundreds, some that will purposely go against the school system. So, fractures in the organizing are expected. However, when it begins taking considerable negative affect on students and their education, any and all resources must be dedicated to rectifying and reinstating high quality organising. It also strains the teachers already demanding job, when they cannot provide the service, they are obligated to offer imply due to a lack of communication. Frustration can get a hold on both student and staff, making the staff workplace and students learning area an undesirable environment

Stakeholders

The most probable stakeholders of this project would be the staff of the school and the students of the school. The staff would use the proposed solution by the utilities it provides. To aid scheduling their timetable and keeping up to date on their co-workers. This program would offer a booking mechanism and also neatly and easily display vital information.

The teachers especially could use this mechanism to book room they would like to use, and could even see what rooms would be free and not free at any and all specific times. This allows them to teach and aid students effectively.

Students will be more indirectly affected. If teachers are unorganised and are unaware of schedule changes the students will lose vital lesson time and knowledge.

Staff other than teachers can also benefit. Such as receptionists if they need to find the location of a teacher or student at specific times. This can avoid time-consuming searching for specific individual. And also, for IT staff When repairing and maintaining equipment, they could avoid interrupting live lessons by using the displayed time tables on the program. This is so they can work safely without students and staff in their way, and the students can gain the most out of their lessons.

Computational solution

There are multiple reasons justifying a computational solution is suitable. Such as the abundant data, constantly required updates to multiple different variables, easy and quick access is necessary throughout the entire school building at all times and also it must support multiple users parallelly.

Using a computational solution, the data can be stored, deleted, sorted, extracted and displayed all electronically. This data could include booked rooms and their times that are booked, a teacher's login details, classroom names and the times they are available throughout the next few weeks. Using SQL databases, data can be effectively organised and easily managed.

The updates (such as greying any dates the current date has surpassed to avoid unnecessary booking records) can be done automatically and constantly, 24/7. And also, when any booking sequences are completed this will simultaneously take effect and show any other users using the program. This makes sure all staff are updated to any updates of classrooms availability. Maintaining clear communication and ceasing any errors of confusion. As long as the program is active. This reduces minute tasks that would've taken staff time to reorganise if, for example, it was on paper.

Due to this system being electronic and computational, Staff could access it from anywhere in the building as long as they have necessary equipment. Such as a computer, laptop, Tablet, mobile device. Any device that could connect to the school's internet, and then can access the school's servers. Or perhaps a less expensive option, designated devices implemented around the school body where teachers can access the program. If the program can be accessed via internet, the program can be accessed off school premises, so staff can continue planning their schedule at home. Whereas a non-computational solution will not grant such a wide array of possible accessibility.

A program will be able to display the same data to multiple devices and multiple users. This inhibits queuing to access data, alter booked rooms and overall increase efficiency. Staff on opposite sides of the buildings and in separate rooms can be updated to new information and even alter information at the same time. There is a risk of users clashing, meaning trying to alter the same data at the same time. In this case, staff trying to book the same room at the same data, however protocols and precautions can overcome this error. If this was on paper for example, you would need to print out the information for each staff, and it can become outdated rather quickly

Existing similar solutions

Many computational solutions already exist, such as ARBOR, RM Inegris, Bromcom, Cmis, Pupil Assets, Isams, ScholarPack... However, the school I currently Attend to uses one named SIMS.

SIMS was Initially developed in 1982-1993 by Philip Neal, a teacher from Lea Manor High School. Then the Schools local education authority furthered developed the software and it began being used in schools by 1984. In 1988 SIMS limited was founded. And is still being updated to this day. The first school MIS (Management Information System) was developed in 1978. 43 years ago.

At seeing SIMS for the first time, one thing stuck out. Incredibly new user non-friendly. Having many hours of experiencing a variety of gaming software's and consoles, detecting user -friendliness is incredibly important for any type of software to be a successful program no matter what the use and objective it has. SIMS seems to have an incredibly complex UI. This UI may be precise and offers a variety of tools, to a new user they would be overwhelmed. I have seen this first-hand with experienced teachers not trained in such utilities. Where staff who understand SIMS now has to dedicate time to teach and explain this puzzle.

Some are major world-wide companies. SIMS is used in 19,000 schools over 49 countries. Arbor is used in over 10,000 schools. And Bromcom works with over 800 schools, 60 MATs (Multi Academy Trust) and 20 LA (local Authorities) Each have an advantage over another, and some specialised in different levels of education. For example, either primary or secondary school. Maybe even colleges and Unis.

Seeing each program briefly, clear vital key points stand out. UI and databases. How well is it displayed? Is data clearly shown? How easy is the program to navigate? If looking a specific variable, could an unexperienced user locate it? The UI is the first impression users will receive, so it is without a doubt essential that it is simple to navigate and displays data clearly. The databases are also incredibly important. Because these will dictate how all the data in the system is managed. And if the data is managed poorly, it will be a poor management information system. SQL is a very popular choice. However, they are not always the most suitable, for example in hierarchical data storages.

Schools have been around since the 1600s, management information systems (like SIMS) have been around for only 43 years. When researching this I saw how differently these products were, and overall, it is not really a surprise. This is a new concept. And we are trying to implement this new concept with a system we have had and refined for the last 400 years. We do not know the best ways yet; it is still up for experimenting and debate. There are no set rules for what you need and cannot have.

Features of proposed solution

There are essential features that are a MUST and some features that are not essential however still useful.

1.) THE BOOKING TOOL – The core of this entire program, even the name is Classroom Availability. It shows all classrooms schedule, what times on what days are they free / not free. This is the fundamental backbone of the entire project, so the booking tool is paramount in its functional ability. The booking utility must allow a LOGGED IN USER to change any classrooms availability value to unavailable at any session at any date. However, any chosen dates and sessions to “book” must already be available and the chosen date must not be chronologically behind the current date. There must be a clear list (visible to the user) of already chosen dates the user has picked.

2.) UI SHOWING CLASSROOM SCHEDULE – Although the booking system is paramount so staff (teachers) can update the data, displaying the data is also vital for keeping the STAFF up to date. This data must be shown clearly and simply and organised effectively. A neat UI will give the user a first good impression of a high-quality service. They will not feel overwhelmed and be incentivised in the future to utilise the program.

3.) PROGRAM AVAILABLE EVEN NOT LOGGED IN – The program must be available to access even when not logged in, this can allow for non-teaching staff (IT and/or receptionists) to access the classrooms schedule without any login details to input/remember. Logging in only privileges the current use to book / view their bookings/ cancel any of their boking records. While no non-teaching staff will be looking to alter data, they still may find accessing the current data to be useful.

Updating the data and keeping the staff updated is the foundation of which this solution must be built o, however additional décor can be implemented to increase the worthiness of the program, such as:

Multiple ways to visualise classroom availability. As you can see specific floors, ground floor or second floor, or perhaps a search bar to locate a specific classroom at the chosen date/ session. A list of every single classroom available on the date, split into sessions. Or maybe classrooms are split into their designated faculties (however this may work for my specific school I attend; this may not be practical in others.

Another additional component can be a page of all booking records the current user has under their name. A requirement for this is the user must be logged in. And only the user can see their own bookings. Perhaps a choice to cancel/delete any of their booking records.

Limitations of proposed solution

Although the most suitable and preferable solution to the school's logistic needs, like any system it has limitations / flaws. One main disadvantage is this solution cannot be accessed and utilised without the correct equipment. This equipment is electronic devices. Their moveability can be limited due to wires and needing a constant connection to power. Any battery sufficed equipment can run out and once they have shut down, the program cannot be accessed. While if the solution was on, for example, paper, this disadvantage does not exist. Another is budgeting. The equipment just mentioned comes with considerably financial expense. Along with the bill of increased electrical consumption.

Another limitation is inexperience and lack of teaching of how to use new electronic solutions. For example, new teaching staff are given advice and aid by more experienced staff. This is a common practice and is abnormal for new teaching staff to go through training days. There is no such training system for teaching staff how to use new electronic devices and services. Therefore, if an electronic device / solution is introduced in the education system, another system of training staff how to use this new concept must accompany this.

Software and hardware requirement

Hardware requirements can range from any and all electronic devices that can access the internet / school virtual databases. The new program would be most suitably based on a server, so it can access at any time by multiple users, using a star topology. Therefore, a router would be required that can maintain multiple wireless connections. Possible wired connection can be a positive bonus but wireless connections are paramount. This is so the program can be accessed throughout the entire stretch of the building to increase optimal efficiency for staff.

The software requirements would be a program that is compatible with minimally, windows operating system. This OS is the most commonly used in education systems due to its versatility and also less expensive compared to its most threatening competitor MACOS. Being compatible with MACOS can be a benefit however due to this system being used specifically for educational purposes, it should focus on the Windows OS. The software should be based on the server, and is accessible by a clear icon. All software pages and documents withholding the data for the program to use must be out of the users' access and view to show high professional standards and also so the program cannot be intentionally / unintendedly tampered with.

ANALYSIS SUCCESS CRITERIA

- The user must Be able to login
 - User interface must present easy access to login utility
- System must validate any login details entered once user requests to login
 - System must deny any invalid login details
- System accepts valid login details and logs user into correct user profile.
 - Once logged in, user has access to log out

- The user can alter / decide what date they are presented
- The current date that is currently selected is shown clearly to user
 - User has access to calendar utility to change date
- The date can be changed to any weekday in the year 2021

- All classrooms are shown clearly
 - The classes are differentiated by faculties
- A page shows all classes available on the selected day grouped by session time
 - All classrooms schedule on selected date is clearly shown.

- Logged in User can book a room
 - A user can select a room and choose a session time
- User can choose the dates to book the selected room at the selected session time
 - A page will show all bookings under the currently logged in user

PROJECT DESIGN

Decomposing the problems

The problem is the software. And the main sections are the software's:

User interface / stakeholder data input / Utilities.

Software: User Interface

SIMS GUI seems unnecessarily messy and complex. Multiple buttons and pages overwhelming users. The data is inefficiently presented to the user. This puts off newcomers to the software and continually even confusing experienced staff. The UI does not need to be complex. Using the structure of the schools itself from a bird's eye view you can use it for a GUI. The whole Ui will be created using the python module TKINTER.

Software: Stakeholder data input

All staff from the school will have an individual login. They will have a username and password only to them. This will grant them access to the utilities booking and cancelling bookings of classrooms. Also, the data they will have to enter which classroom they want to book, and how many weeks preceding the current one they want that classroom to remain booked. When they want to cancel a booking of a classroom, they must select which classroom and how many bookings they want to cancel.

There could also possibly be a search tool. To find the list of all free / booked classroom at a certain time and/or date. Or find a single classrooms status for a specific week. So, the user must input the required variables for this search tool to discover results.

Software: Utilities

The two main prioritized utilities are the Booking tool and the Cancelling tool. These are quite self-explanatory. The booking utility allows the user to alter a classrooms status from **AVAILABLE** to **UNAVAILABLE**. This utility can be used in multiple different ways. You can book a classroom just as a one-off for that day and time. Or you can book that class for specific number of future weeks (including that current week). Or book it for every week in the future until further notice. The various ways you can use this tool is of high importance. The cancelling tool will be used to change the classroom status from **UNAVAILABLE** to **AVAILABLE**. Again, various ways to use this tool, cancel just the one time or even follow a pattern. For example, cancelling the booked room for every 2 weeks (this will also be available with the booking tool)

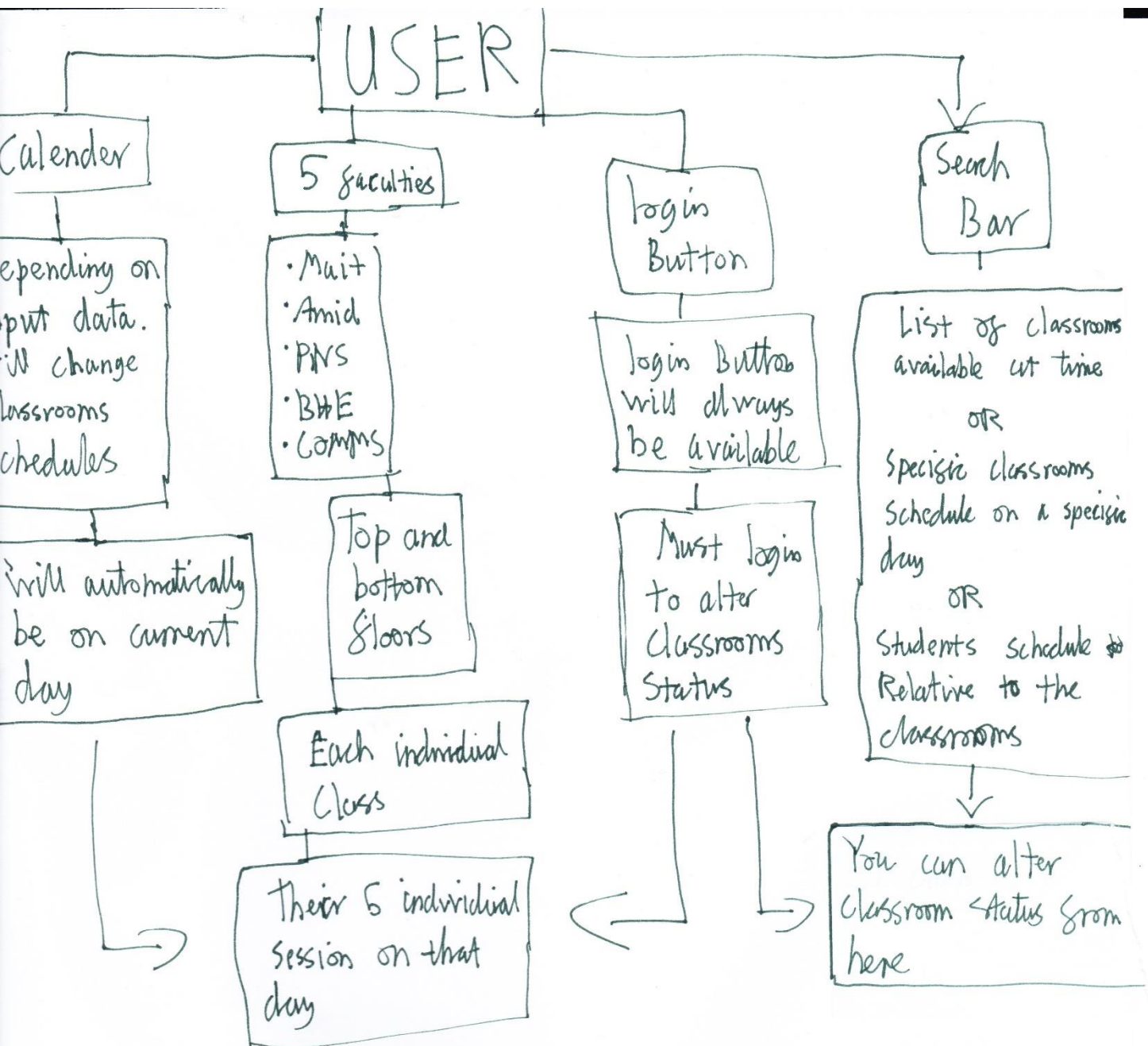
The search tool is not for certain it'll make it to the final product yet however it is most likely. It will be a way for user to find certain classrooms status at specific times and days. Or even a certain class or single student to where they will be at certain times.

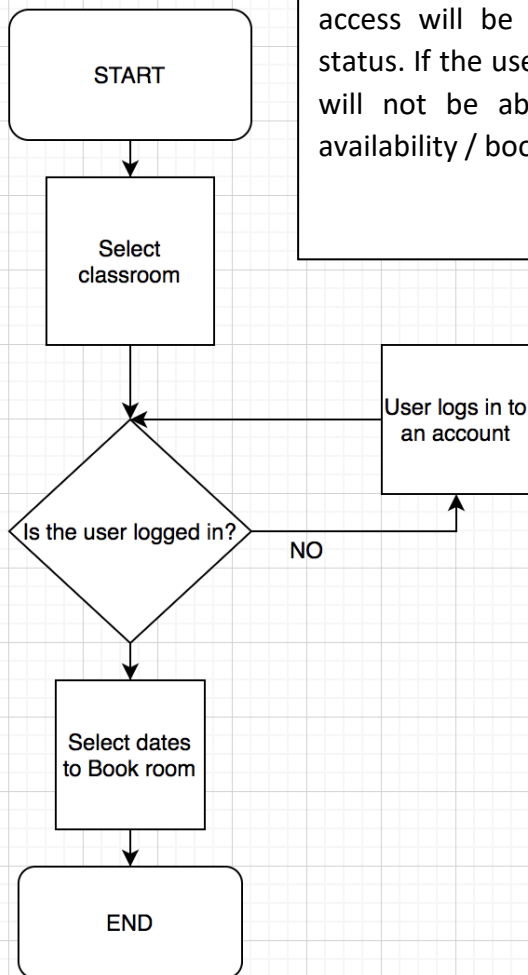
Describing the Solutions

The structure of the solution is quite diverse. The structure is a whole new program. This can seem quite radical and unnecessary for a few problems that the current system integrated at CBA holds. However, to justify the need for anew system are these arguments.

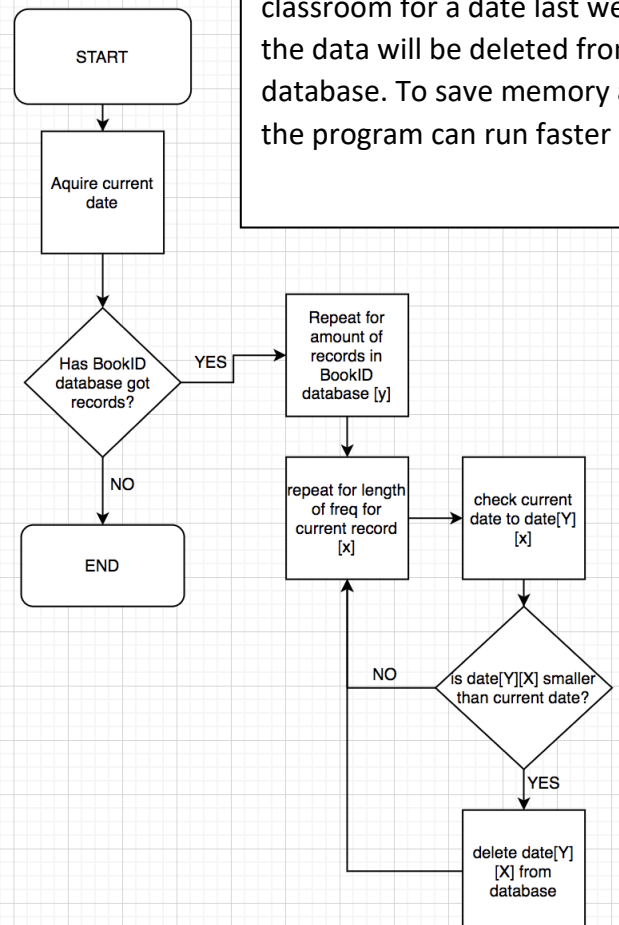
- SIMS cannot be altered and edited to fit the staff's requirements and rectify problems because it is closed software. The code cannot be accessed and legally belongs to the company. So, SIMS software cannot be **LEGALLY** edited.
- Another program may be purchased and used at the at the school. However, this is no guarantee it will solve the problems that the current one has. The only guarantee is the expense the school will need to pay for this risky alternative. A whole new code created from scratch will come with no expense and 100% guarantee it will meet the requirements.

Flow charts





Here I have an idea of how booking access will be dictated by logged in status. If the user is not logged in, they will not be able to alter classroom availability / book classrooms.



I have another mechanism for searching and deleting outdated bookings. If a user has booked a classroom for a date last week, the data will be deleted from the database. To save memory and the program can run faster

CODE

Thinter

Pre-made calendar code should be used

→ Make it compatible to work with all code

↓
Need some sort of timer for code, to keep updated.

login

will open a separate smaller window when ~~click~~ clicked

!
If login successful, window closes and opens to Book rooms.

5 faculties

All in individual classrooms on top and bottom level

Schedule of the classrooms & sessions.

If available, user is logged in, user can book, turns to unavailable.

If Unavailable cannot book or alter

Will automatically convert csv side to textfile. This task will be used for program

creating/updating classroom schedule

↓
textfile will be used to alter classrooms sessions are

Available or Unavailable

Can book for single/multiple sessions.

will alter textfile when booking

↓
when booking time expires, textfile will revert back

Data Structures

The classrooms timetables will be stored in a database. A separate text file that will be accessed by the python code. However, it is unclear whether there will be a 2nd separate text file with the exact same data and when python alters the data, this 2nd text file will be the one it will alter. The **DATA TYPE** will be strings in this database. The columns act as a single session in the school day. So, there will be a total of 25 columns (5 sessions to each weekday). If the classroom name is in the column (E.G, ML5) it will be classed as AVAILABLE. If it is not found in the column it is Unavailable at that specific time. To make it easier for me to edit and work with the database, I made the database in a numbers document [An apple application.

Alternative app to Excel from Microsoft]. So, the numbers file will be converted by the same python code before use

Testing

The test data does not need to be based on any real data. All it needs to be is inserted in the same structure. From a numbers document OR text file. However, the school withholds a list of some sort of free classrooms per session, so the test data may originate from there. However, there is no need for a strict rule of where the test data comes from.

Frequency of testing is not strict. The demand for the program will not alter, and requirements will not change. However, to keep the program simple and User friendly, when the programs hits a certain point of fully functioning. Testing should be conducted on a variety and high number of staff. Possibly all staff due to the relatively small number and the code can be transmitted via email so feedback can be quickly and easily gathered.

DEVELOPMENT

Fundamentals

Firstly, to initialise Tkinter, you have to create a TK root widget. Which acts as a window with the title bar and other decoration provided by the window manager. The root widget must be created before any other widget, and there can only be one root widget. We need to import the Tkinter module into the python file to use it in the library. We import it and create a variable to represent the Tkinter module, here I created the variable 'tk'. And after I state root = tk.Tk(). This states root = the root widget of the Tkinter module.

```
import tkinter as tk root = tk.Tk()
```

Using the root widget at the beginning of the program, you can set the dimensions, title, font of all text and even background of the Tkinter window. Such as:

```
root.geometry('1920x2000')  
root.option_add("*font", "lucida 14" )  
root.title("Mainpage")  
root.configure(background='black')
```

VERSION 1, User Interface Development:

XandyY

First was creating the Tkinter UI. The first page I wanted to create was the main page of the birds' eye view of the school. This was rather simple with a few boxes to create a rough outline of the school. One main corridor in the middle and 6 faculties. However, I had a problem trying to structure the boxes on the screen. An interesting development I found about Tkinter was the columns and rows do not exist until an entity lies within said column / row. So, to counter this I created an X and Y graph to help me structure my UI. I got the idea in one of my math classes when working with 2D diagrams. This creates a useful tool when I'm placing and altering the position of any Tkinter object on the screen. When I choose the entities row and column value, I can use my X and Y diagram to aid me with accurate placements and also it creates sufficient space to utilise.

```
root_label = Label(root, text="column: 0")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=0)
root_label = Label(root, text="column: 1")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=1)
root_label = Label(root, text="column: 2")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=2)
root_label = Label(root, text="column: 3")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=3)
root_label = Label(root, text="column: 4")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=4)
root_label = Label(root, text="column: 5")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=5)
root_label = Label(root, text="column: 6")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=6)
root_label = Label(root, text="column: 7")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=7)
root_label = Label(root, text="column: 8")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=8)
root_label = Label(root, text="column: 9")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=9)
root_label = Label(root, text="column: 10")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=10)
root_label = Label(root, text="column: 11")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=11)
root_label = Label(root, text="column: 12")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=12)
root_label = Label(root, text="column: 13")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=13)
root_label = Label(root, text="column: 14")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=14)
root_label = Label(root, text="column: 15")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=15)
root_label = Label(root, text="column: 16")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=16)
root_label = Label(root, text="column: 17")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=17)
root_label = Label(root, text="row: 1")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=1, column=0)
root_label = Label(root, text="row: 2")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=2, column=0)
root_label = Label(root, text="row: 3")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=3, column=0)
root_label = Label(root, text="row: 4")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=4, column=0)
root_label = Label(root, text="row: 5")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=5, column=0)
root_label = Label(root, text="row: 6")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=6, column=0)
root_label = Label(root, text="row: 7")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=7, column=0)
root_label = Label(root, text="row: 8")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=8, column=0)
root_label = Label(root, text="row: 9")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=9, column=0)
root_label = Label(root, text="row: 10")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=10, column=0)
root_label = Label(root, text="row: 11")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=11, column=0)
root_label = Label(root, text="row: 12")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=12, column=0)
root_label = Label(root, text="row: 13")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=13, column=0)
root_label = Label(root, text="row: 14")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=14, column=0)
root_label = Label(root, text="row: 15")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=15, column=0)
root_label = Label(root, text="row: 16")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=16, column=0)
root_label = Label(root, text="row: 17")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=17, column=0)
root_label = Label(root, text="row: 18")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=18, column=0)
root_label = Label(root, text="row: 19")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=19, column=0)
root_label = Label(root, text="row: 20")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=20, column=0)
```

```
root_label = Label(root, text="column: 20")
root_label.configure(width=7, height=1, background='black', foreground='black')
root_label.grid(row=0, column=20)
```

Here we see one entity in the whole X and Y function. So, we give the object a name, 'root_label'. We class the object as a Label on the first line Infront of the brackets. Inside the brackets, we connect it to the root of Tkinter. That we simply call 'root'. Along with the text 'column:20' that will be automatically in the middle of the object. Anything in the brackets MUST be separated by a comma.

```
root_label = Label(root, text="column: 20") ]
```

On the second line, we define the size and colours of the object. First you must define the object you are talking about by root_label. Configure, add. configure on the name. Then place brackets right next to the configure (). Here you can define width=7, and height=1, by the word then '=' then a number for the value. Again, separate all of these characteristics of the object by a comma. With background and foreground, the background sets the colour of the object, and foreground sets the colour of any text in the object. You can set the colour by ASCII Hexadecimal code, or by the name of the code but if you use the name you need to put the name in ". Still separate the values by commas.

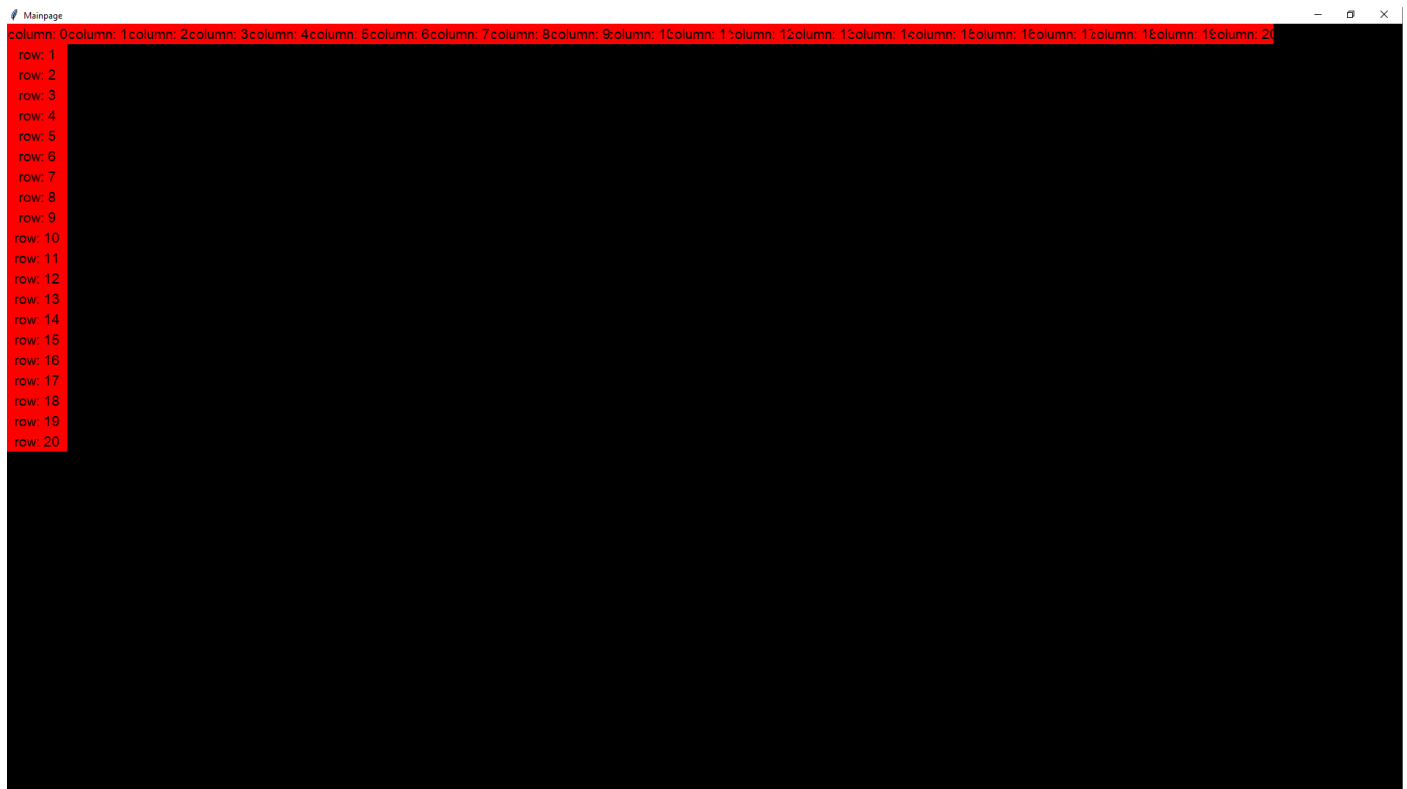
```
root_label.configure(width=7, height=1, background='black', foreground='black')
```

The third and final line for this object defines the objects' location on the window, here with the use of other objects. A row or column does not exist until a widget exists in the said column / row. The column / row is only as

wide or as tall as the widest / tallest widget in the column / row. So this widget has been defined in the 1st row (row 0) and column 20, as the text says column 20 this widget will create and define where the 20th column is. Yewt again, divide values by a column and define the column and row as you define width and height, with an equals sign '=' and a number.

```
root_label.grid(row=0, column=20)
```

Results: A 1920 x 2000 window page open the background as black and the XY widgets. Named Mainpage



Basis of main page

Now I have created an environment to effectively create a UI. This page will connect all pages with each other. This is where all users will be brought to when first entering the program. It will withhold a simplified bird's eye view of the school's layout plus any and all other tools. Such as a button to login, a calendar, a button to see all available classrooms, and any other tools and pages I will choose in the future. Firstly, create

```
## corridor of school
corridor1 = Text(root,width=15, height=10,borderwidth=0,highlightthickness=0)
corridor1.configure(background='grey')
corridor1.grid(row=4,column=4)

corridor2 = Text(root,width=15, height=10,borderwidth=0,highlightthickness=0)
corridor2.configure(background='grey')
corridor2.grid(row=5,column=4)

corridor3 = Text(root,width=15, height=10,borderwidth=0,highlightthickness=0)
corridor3.configure(background='grey')
corridor3.grid(row=6,column=4)

### MAIT
MAITbutton = Button(root,text="MAIT",height=5, width=20,highlightbackground="orange", fg="Black", highlightthickness=20)
MAITbutton.grid(row=5, column=5)
MAITbutton.configure(background='dark orange')

### COMMS
COMMSbutton = Button(root,text="COMMS", height=5,width=20,highlightbackground="red", fg="Black", highlightthickness=20)
COMMSbutton.grid(row=5, column=3)
COMMSbutton.configure(background='red')

### AMID
AMIDbutton = Button(root,text="AMID", height =5,width=20,highlightbackground="blue", fg="Black",highlightthickness=20)
AMIDbutton.grid(row=4, column=5)
AMIDbutton.configure(background='blue2')

### BHE
BHEbutton = Button(root,text="BHE", height =5,width=20,highlightbackground="green", fg="Black", highlightthickness=20)
BHEbutton.grid(row=6, column=5)
BHEbutton.configure(background='green2')

### PNS
PNSbutton = Button(root,text="PNS", height =5,width=20,highlightbackground="purple", fg="Black", highlightthickness=20)
PNSbutton.grid(row=6, column=3)
PNSbutton.configure(background='purple')
```

the MAIT, COMMS, BHE, PNS and AMID widget along with the corridor. Instead of Label outside the brackets you put Button.

Buttons and commands:

With Tkinter button entities you can connect a function to them. So, when the user 'clicks' on the button the function will be executed. This can be connected so it creates the illusion of a new page. But really it will clear the current Tkinter page and rewrite other entities on the refreshed page.

```
def clear_tkinter():  
    list = root.grid_slaves()  
    for l in list:  
        l.destroy()
```

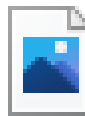
This command will be the foundation of navigating through my program, and the simple buttons will make it clear to the user how to navigate since text can also be inputted in the buttons.

```
### MAIT  
MAITbutton = Button(root, command=clear_tkinter, text="MAIT", height=5, width=20, highlightbackground="orange", fg="Black", highlightthickness=20)  
MAITbutton.grid(row=5, column=5)  
MAITbutton.configure(background="dark orange")
```

Eagle eye view of the school:

Using labels and buttons, I created a simplistic above view of the school with the main corridor as grey labels and six coloured buttons to act as faculties. I coloured the buttons as the same colour as the faculties in real life, and also the name of the faculty was applied in the inside of the buttons. Using the XandY diagram to place and build the structure. The initial way I began placing entities on the screen was by "Name_of_button.grid(row=x, column=y)".

The value of x depended on the row box I wanted to line it up with, and also y depended on the column box.



Media1.mp4

Faculty page

To navigate to the faculty page, you need to selected one of the 5 buttons that were used to create the simplified school overview. The buttons will need to function to refresh the screen and then begin creating a new page. This can be done with a variable. So, you run the function of clearing the page and depending on the button you selected. A variable will also go through to this function, and depending on this variable value another function will run. This is needed due to the BHE button and MAIT button will not lead to the same page, so their variable will differ.

```
### MAIT  
MAITbutton = Button(root, text="MAIT", command = btn_MAIT, height=5, width=20, highlightbackground="orange", fg="Black", highlightthickness=20)  
MAITbutton.grid(row=5, column=5)  
MAITbutton.configure(background="dark orange")  
### COMMS  
COMMSbutton = Button(root, command = btn_COMMS, text="COMMS", height=5, width=20, highlightbackground="red", fg="Black", highlightthickness=20)  
COMMSbutton.grid(row=5, column=3)  
COMMSbutton.configure(background="red")  
### AMID  
AMIDbutton = Button(root, command = btn_AMID, text="AMID", height =5, width=20, highlightbackground="blue", fg="Black", highlightthickness=20)  
AMIDbutton.grid(row=4, column=5)  
AMIDbutton.configure(background="blue2")  
### BHE  
BHEbutton = Button(root, command = btn_BHE, text="BHE", height =5, width=20, highlightbackground="green", fg="Black", highlightthickness=20)  
BHEbutton.grid(row=6, column=5)  
BHEbutton.configure(background="green2")  
### PNS  
PNSbutton = Button(root, command = btn_PNS, text="PNS", height =5, width=20, highlightbackground="purple", fg="Black", highlightthickness=20)  
PNSbutton.grid(row=6, column=3)  
PNSbutton.configure(background="purple")
```

```
def btnn_COMMS():
    global currentfac,currentdate
    currentfac = 1
    create_faculty('red','CL','CU','COMMS',1,2,currentdate)

def btnn_MAIT():
    global currentfac
    currentfac = 2
    create_faculty('orange','ML','MU','MAIT',3,4,currentdate)

def btnn_BHE():
    global currentfac
    currentfac = 3
    create_faculty('green2','BL','BU','BHE',5,6,currentdate)

def btnn_PNS():
    global currentfac,currentdate
    currentfac = 4
    create_faculty('purple','PL','PU','PNS',7,8,currentdate)

def btnn_AMID():
    global currentfac,currentdate
    currentfac = 5
    create_faculty('blue','AL','AU','AMID',9,10,currentdate)
```

The global variables will be explained at a different date. However, the main part is that each button goes to the same function but with different variables. This increases efficiency, less amount lines of code, the better.

Each function has slightly different arguments when initiating the function

The function of clearing out the page is at the start of the create_faculty function.

```
def create_faculty(corridor_colour,bottom_classes,upper_classes,facultyname,VALUE,VALUE2,currentdate):
    list = root.grid_slaves()
    for l in list:
        l.destroy()
    root.geometry('1920x2000')
    root.title('Mainpage'+facultyname)
    XandY()

    currentdate_label = Label(root,text='currently selected date: '+currentdate,background='black',foreground='white',height=2,width=60,font=('lucida',30))
    currentdate_label.grid(row=5,column=8,columnspan=20)

    ## faculty lower central area
    MainCorridor = Label(root,width=10,height=11,highlightthickness=0)
    MainCorridor.configure(background=corridor_colour)
    MainCorridor.grid(row=4,column=4)

    MainCorridor = Label(root,width=10,height=11,highlightthickness=0)
    MainCorridor.configure(background=corridor_colour)
    MainCorridor.grid(row=5,column=4)

    MainCorridor = Label(root,width=10,height=11,highlightthickness=0)
    MainCorridor.configure(background=corridor_colour)
    MainCorridor.grid(row=6,column=4)

    ## faculty upper central area
    MainCorridor = Label(root,width=10,height=11,highlightthickness=0)
    MainCorridor.configure(background=corridor_colour)
    MainCorridor.grid(row=4,column=8)

    MainCorridor = Label(root,width=10,height=11,highlightthickness=0)
    MainCorridor.configure(background=corridor_colour)
    MainCorridor.grid(row=5,column=8)

    MainCorridor = Label(root,width=10,height=11,highlightthickness=0)
    MainCorridor.configure(background=corridor_colour)
    MainCorridor.grid(row=6,column=8)

    ## faculty lower classrooms
    classONE = Button(root,text=bottom_classes + "1",height=10,width=6,command = partial(CLASSONE_btnn,VALUE))
    classONE.grid(row=6,column=3)
    classTWO = Button(root,text=bottom_classes + "2",height=10,width=6,command = partial(CLASSTWO_btnn,VALUE))
    classTWO.grid(row=5,column=3)
    classTHREE = Button(root,text=bottom_classes + "3",height=10,width=6,command = partial(CLASSTHREE_btnn,VALUE))
    classTHREE.grid(row=4,column=3)
    classFOUR = Button(root,text=bottom_classes + "4",height=4,width=6,command = partial(CLASSFOUR_btnn,VALUE))
    classFOUR.grid(row=3,column=3)
    classFIVE = Button(root,text=bottom_classes + "5",height=4,width=6,command = partial(CLASSFIVE_btnn,VALUE))
    classFIVE.grid(row=3,column=5)
    classSIX = Button(root,text=bottom_classes + "6",height=10,width=6,command = partial(CLASSSIX_btnn,VALUE))
    classSIX.grid(row=4,column=5)
    classSEVEN = Button(root,text=bottom_classes + "7",height=10,width=6,command = partial(CLASSSEVEN_btnn,VALUE))
    classSEVEN.grid(row=5,column=5)
    classEIGHT = Button(root,text=bottom_classes + "8",height=10,width=6,command = partial(CLASSEIGHT_btnn,VALUE))
    classEIGHT.grid(row=6,column=5)

    ## comms upper classrooms
    CUONE = Button(root,text=upper_classes + "1",height=10,width=6,command = partial(CLASSONE_btnn,VALUE2))
    CUONE.grid(row=6,column=7)
    CUTWO = Button(root,text=upper_classes + "2",height=10,width=6,command = partial(CLASSTWO_btnn,VALUE2))
    CUTWO.grid(row=5,column=7)
    CUTHREE = Button(root,text=upper_classes + "3",height=10,width=6,command = partial(CLASSTHREE_btnn,VALUE2))
    CUTHREE.grid(row=4,column=7)
    CUFOUR = Button(root,text=upper_classes + "4",height=4,width=6,command = partial(CLASSFOUR_btnn,VALUE2))
    CUFOUR.grid(row=3,column=7)
    CUFIVE = Button(root,text=upper_classes + "5",height=4,width=6,command = partial(CLASSFIVE_btnn,VALUE2))
    CUFIVE.grid(row=3,column=9)
    CUSIX = Button(root,text=upper_classes + "6",height=10,width=6,command = partial(CLASSSIX_btnn,VALUE2))
    CUSIX.grid(row=4,column=9)
    CUSEVEN = Button(root,text=upper_classes + "7",height=10,width=6,command = partial(CLASSSEVEN_btnn,VALUE2))
    CUSEVEN.grid(row=5,column=9)
    CUEIGHT = Button(root,text=upper_classes + "8",height=10,width=6,command = partial(CLASSEIGHT_btnn,VALUE2))
    CUEIGHT.grid(row=6,column=9)
```

Since there is two floors in each faculty there needs to be two corridors create.

This current date will be discussed later

Here is the actual classroom, each floor gets 8 classes. The row and column are set precisely so they connect to the central corridor. As you can see, they also have a command with arguments (that you need to use partial with Tkinter button widgets.

Eagle – eye view of faculty:

Like the school on the main page, the faculties will also be seen from above and will also share the basic shape. A long corridor in the middle, the Classes all around the middle corridor. Faculties have different classrooms depending on the floor. So, this means two separate views per faculty, the top and bottom floor. Each will have identical corridor but different classes. Each floor will consist of 8 classrooms. Since they have their own unique arguments going into the function depending what faculty button they chose, one also dictates the colour of the corridor. Following the colour theme of the chosen faculty.

The names of the classrooms are the same as the real school. ML1 = M(Mait) L(Lower) 1(One), this tells you that this classroom is in MAIT, Lower floor and 1. The classes are numbered in increasing interval clockwise starting in the bottom left corner of the rectangle.

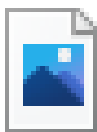
I also created a back button to go back to the main page. So, I had to put the main page code in a function. Along with the refresh page code.

```
def mainpage():
    list = root.grid_slaves()
    for l in list:
        l.destroy()
```

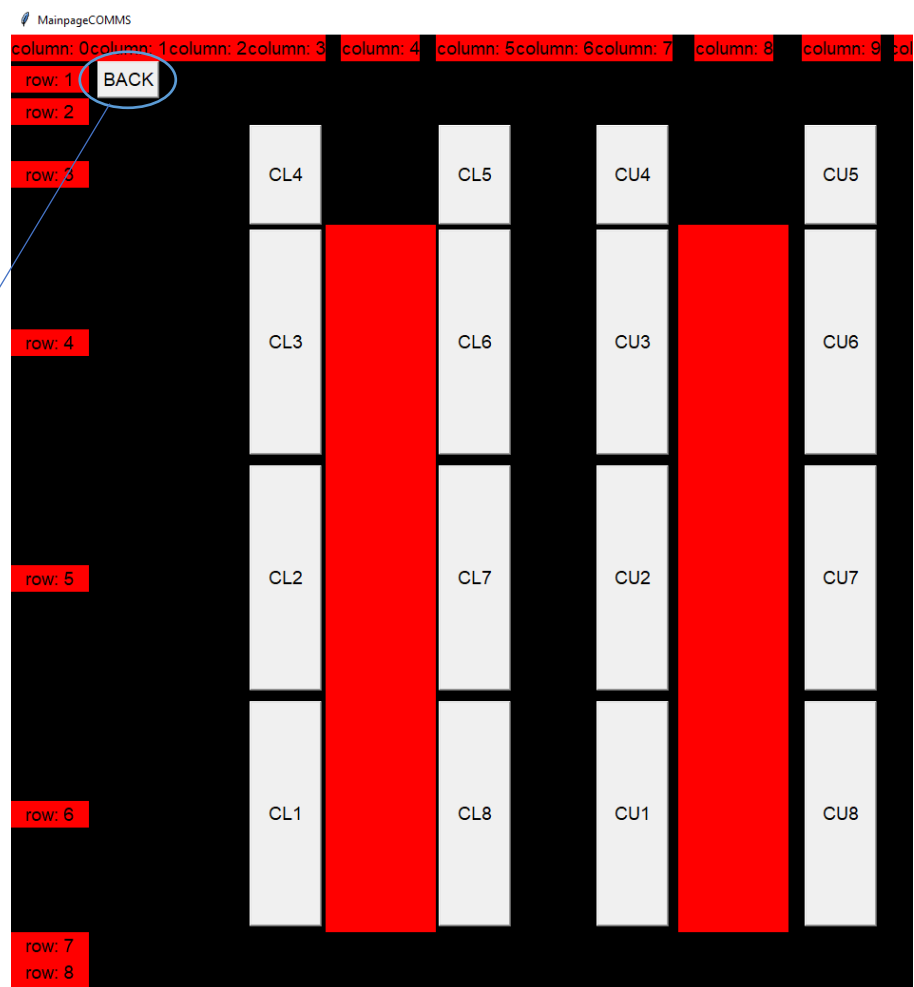
So then I created a widget in the faculty page.

```
## back button
BACK = Button(root,text="BACK", command = mainpage , height=1 , width=5)
BACK.grid(row=1,column=1)
```

Now you can navigate to and from the main and faculty pages



Media1.mp4

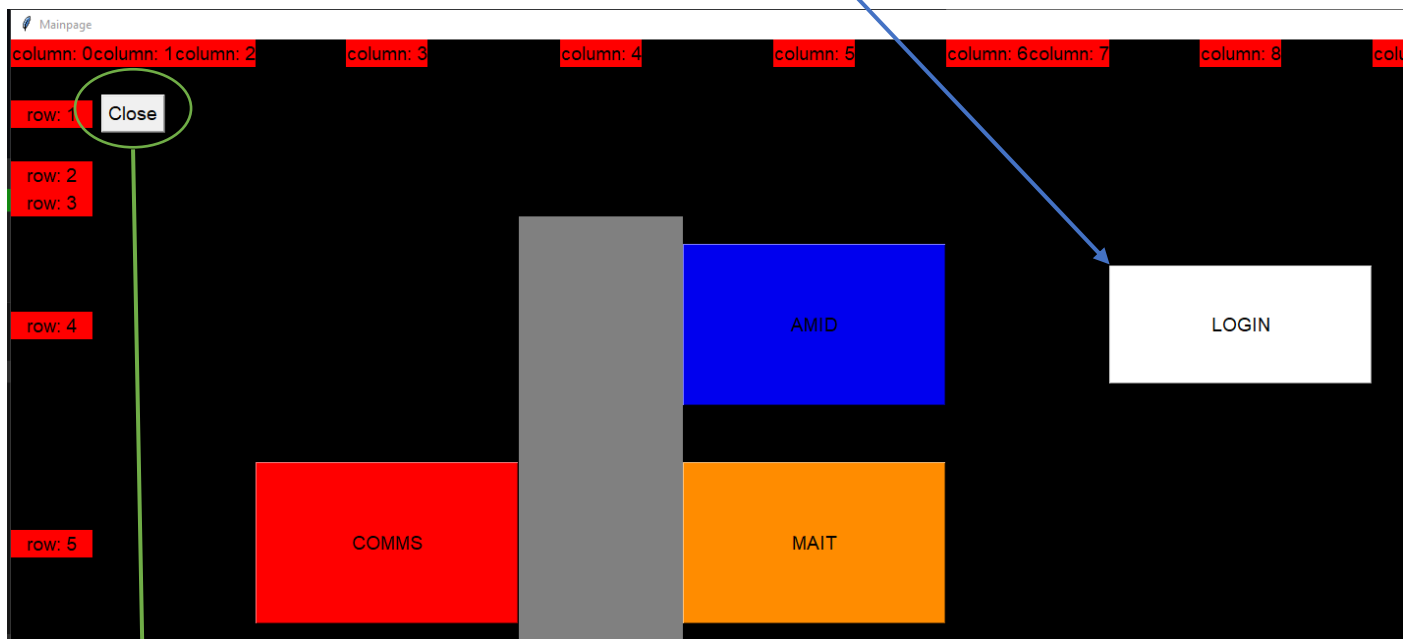


Login UI

For the login page I want a separate window page, a smaller one. We don't need the entire 1920 x 2000 page for the user to type in a username and password to login. Firstly, we need to add a widget on the main page function.

```
## login button
if LOGGED_IN == 0:
    LOGINbutton = Button(root,text="LOGIN", command = partial(btnn_LOGIN,mop),height=3, width=20,highlightbackground="white", fg="Black",
    LOGINbutton.grid(row=4,column=8)
    LOGINbutton.configure(background='white')
```

fg is just abbreviation for foreground



```
## close mainpage
def close_mainpage():
    root.destroy()

BACK = Button(root,text="Close", command = close_mainpage , height=1 , width=5)
BACK.grid(row=1,column=1)
```

A close button was created on the main page. Not back, 'close'. The code `root.destroy()` will cease the root widget we created at the very beginning. This will close and halt the entire Tkinter program.

The command you see above circled in purple is a function this button will commence once selected.



```

def btn_LOGIN(mop):

def validatelogin():
    if username_enter.get() == 'ADMIN':
        restart_FreeRooms_table()
        restart_StaffID_table()
        restart_BookID_table()
        restart_RoomID_table()

    else:
        details = checklogindetails(username_enter.get(), password_enter.get())
        if details == None:
            logged_in = Label(roottwo, text="Incorrect login details", width=20, height=1, highlightthickness=0)
            logged_in.configure(background=dark grey)
            logged_in.grid(row=6, column=3, columnspan=2)

        else:
            LOGGED_IN = 1
            username = details[0]
            Staff_name = details[1]
            if reset_all_classes == 0:
                if resetclassschedule == 0:
                    mainpage()
            list = roottwo.grid_slaves()
            for i in list:
                i.destroy()

            roottwo.geometry(700x200)
            roottwo.protocol("WM_DELETE_WINDOW", disable_event)
            logged_in = Label(roottwo, text="you are now logged in as " + Staff_name, width=50, height=1, highlightthickness=0)
            logged_in.configure(background=dark grey)
            logged_in.grid(row=0, column=0, pady=50)
            logged_in.config(font=("lucida", 20))

            BACK = Button(roottwo, text="CLOSE", command = close_page, height=1, width=5)
            BACK.grid(row=1, column=0)

roottwo = Tk()
roottwo.protocol("WM_DELETE_WINDOW", disable_event)
roottwo.geometry(600x200)
roottwo.option_add("*font", "lucida 14")
roottwo.title("Login")
roottwo.configure(background=dark grey)

username = Label(roottwo, text="username", width=8, height=1, highlightthickness=0)
username.configure(background=dark grey)
username.grid(row=3, column=2)

password = Label(roottwo, text="password", width=8, height=1, highlightthickness=0)
password.configure(background=dark grey)
password.grid(row=5, column=2)

blackblock = Label(roottwo, width=8, height=1, highlightthickness=0)
blackblock.configure(background=dark grey)
blackblock.grid(row=4, column=1)

password_enter = Entry(roottwo, width=20, highlightthickness=0)
password_enter.configure(background=white)
password_enter.grid(row=5, column=3, padx = 10)

username_enter = Entry(roottwo, width=20, highlightthickness=0)
username_enter.configure(background=white)
username_enter.grid(row=3, column=3, padx = 10)

## login button
login_btn = Button(roottwo, text="login", command = validatelogin, width=5, height=1)
login_btn.configure(background=dark grey)
login_btn.grid(row=6, column=2)

## close login page
def close_page():
    roottwo.destroy()
## back button to close login window
BACK = Button(roottwo, text="BACK", command = close_page, height=1, width=5)
BACK.grid(row=1, column=1)

```

Here we create an entire new window. To do this we must create a new root widget, here we call it roottwo. All widgets with roottwo will be placed on this new window you can see the creation of this new window plus the dimensions, font, title and background at **1.** A new line of code in this red box is

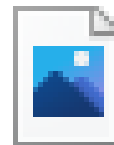
```
roottwo.protocol("WM_DELETE_WINDOW", disable_event)
```

This line stops the close button in the top right of the window from working. So, you must use the back or close buttons to navigate through. This can also help with the program keeping track of what windows are open / close.

When the login button is selected, another function initialises called 'validatelogin'. This uses a separate python file and a separate database. This will be explained later.

I also created an ADMIN function, in **2.** so, all databases Are reset, these databases will be explained later.

We've talked about button and label widgets, now we see Entry widgets. These widgets you can type in them. And later you can retain what was typed in there by entry_widget_enter.get(). This is a variable.



entry widget
example.mp4

Available classes page

```
##### ALL AVAILABLE CLASSES #####
def Allavailableclasses():
    list = root.grid_slaves()
    for l in list:
        l.destroy()
    global currentdate,resetclasschedule,reset_all_classes,listoffreeclasses,calenderopen
    resetclasschedule = 0
    reset_all_classes = 1

def create_allclass_lists(listoffreeclasses):
    global D,M
    RoomNames = ["AL01","AL02","AL03","AL04","AL05","AL06","AL07","AL08",
        "AU01","AU02","AU03","AU04","AU05","AU06","AU07","AU08",
        "BL01","BL02","BL03","BL04","BL05","BL06","BL07","BL08",
        "BU01","BU02","BU3","BU04","BU05","BU06","BU07","BU08",
        "CL01","CL02","CL03","CL04","CL05","CL06","CL07","CL08",
        "CU01","CU02","CU03","CU04","CU05","CU06","CU07","CU08",
        "ML01","ML02","ML03","ML04","ML05","ML06","ML07","ML08",
        "MU01","MU02","MU03","MU04","MU05","MU06","MU07","MU08",
        "PL01","PL02","PL03","PL04","PL05","PL06","PL07","PL08",
        "PU01","PU02","PU03","PU04","PU05","PU06","PU07","PU08"]
    setcolumn = 0
    sessiontime=["8:40-9:40","9:40-10:40","11:30-12:30","12:30-14:00","14:00-15:00"]
    colours = ["red","blue","green","orange","purple"]
    for x in range(5):
        row = 2
        session_time = sessiontime[x]
        padx = 0
        session1 = Label(root,text="SESSION"+str(x+1),background = colours[x],height=2,width=10)
        session1.grid(row=1,column=setcolumn)
        for y in range(len(listoffreeclasses[x])-1):
            y = y + 1
            check = 0
            for m in range(len(RoomNames)):
                if RoomNames[m] == listoffreeclasses[x][y]:
                    check = 1
            if check == 1:
                row = row + 1
                FLN = listoffreeclasses[x][y]
                classroom = Button(root,text=listoffreeclasses[x][y],command=partial(book_room,FLN,session_time),background =colours[x],height = 1,width=5)
                classroom.grid(row=row,column=setcolumn)
            if row == 24:
                setcolumn = setcolumn + 1
                row = 2
                padx = 0

root.title("Mainpage , All available classes")
XandY()

## back button
BACK = Button(root,text="BACK", command = mainpage , height=1 , width=5)
BACK.grid(row=0,column=1,)

create_allclass_lists(listoffreeclasses)
```

This one we do need the entire page, 1920 x 2000 to represent all available classes in the 5 sessions of the date selected. At the start of the function, we use the code to wipe off all widgets on the current page, then we create new widgets. Its on the same page so we use the same root. Called 'root'. We also change the title of the page by:

```
root.title("Mainpage , All available classes")
XandY()
```

Also, we need to run XandY function because we got rid of it when we 'refreshed' the page. Also, a back button like we saw in the faculty and login page.

Now, the purple function called 'create_allclass_lists' creates the lists. It does this by using the global variable called listoffreeclasses as an argument. This argument is a 2D array, it has 5 separate lists each are the classes that are free on that corresponding session. First list is the

classes that are free in the first session '8:40-9:40'... I will explain how we get the variable listoffreeclasses later. We check every variable in each list to another list called RoomNames, we do this in a for loop of 5 because there are five sessions so five lists are going to be shown to the user on this new page titled 'Mainpage, All available classes'.

An array called RoomNames, that holds the names of all the classrooms names in the school.

```
def create_allclass_lists(listoffreeclasses):
```

```
RoomNames = ["AL01","AL02","AL03","AL04","AL05","AL06","AL07","AL08",
"AU01","AU02","AU03","AU04","AU05","AU06","AU07","AU08",
"BL01","BL02","BL03","BL04","BL05","BL06","BL07","BL08",
"BU01","BU02","BU03","BU04","BU05","BU06","BU07","BU08",
"CL01","CL02","CL03","CL04","CL05","CL06","CL07","CL08",
"CU01","CU02","CU03","CU04","CU05","CU06","CU07","CU08",
"ML01","ML02","ML03","ML04","ML05","ML06","ML07","ML08",
"MU01","MU02","MU03","MU04","MU05","MU06","MU07","MU08",
"PL01","PL02","PL03","PL04","PL05","PL06","PL07","PL08",
"PU01","PU02","PU03","PU04","PU05","PU06","PU07","PU08"]
```

Here we run this code 5 times
dues to five sessions and five
lists must be created in UI
format

This creates the title of sessions, session 1,
session 2, session 3. They are all on the
same row but different columns. By
changing column value by set columns

```
setcolumn = 0
sessiontime=["8:40-9:40","9:40-10:40","11:30-12:30","12:30-14:00","14:00-15:00"]
colours = ["red","blue","green","orange","purple"]
```

```
for x in range(5):
```

```
row = 2
session_time = sessiontime[x]
padx = 0
```

```
session1 = Label(root,text="SESSION"+str(x+1),background = colours[x],height=2,width=10)
```

```
session1.grid(row=1,column=setcolumn)
```

```
for y in range(len(listoffreeclasses[x])-1):
    y = y + 1
```

```
check = 0
```

```
for m in range(len(RoomNames)):
```

```
    if RoomNames[m] == listoffreeclasses[x][y]:
        check = 1
```

```
    if check == 1:
```

```
        row = row + 1
```

```
        FLN = listoffreeclasses[x][y]
```

```
        classroom = Button(root,text =listoffreeclasses[x][y],command=partial(book_room,FLN,session_time),background =colours[x],height = 1,width=5)
```

```
        classroom.grid(row=row,column=setcolumn)
```

```
    if row == 24:
```

```
        setcolumn = setcolumn + 1
```

```
        row = 2
```

```
        padx = 0
```

Here we compare the entire RoomNames to a single variable
in one of the 5 lists in the argument listoffreeclasses.

X = the one of the five lists

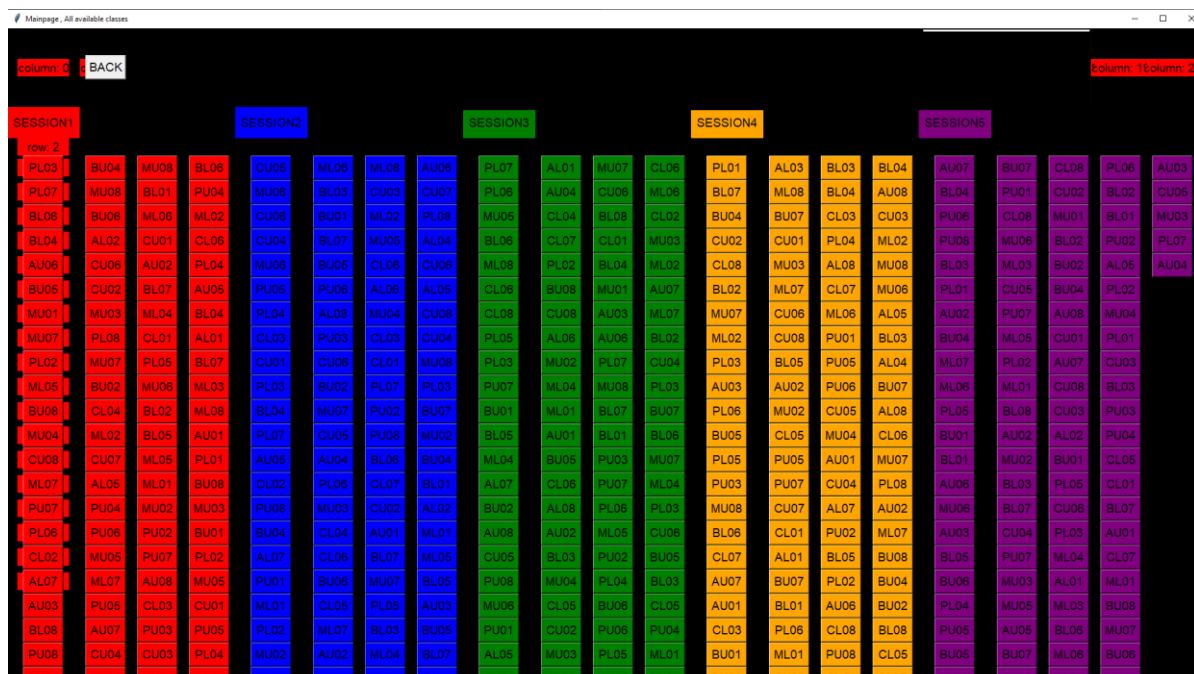
Y = one of the varailbes in the current list (x)

This is where the widgets are created

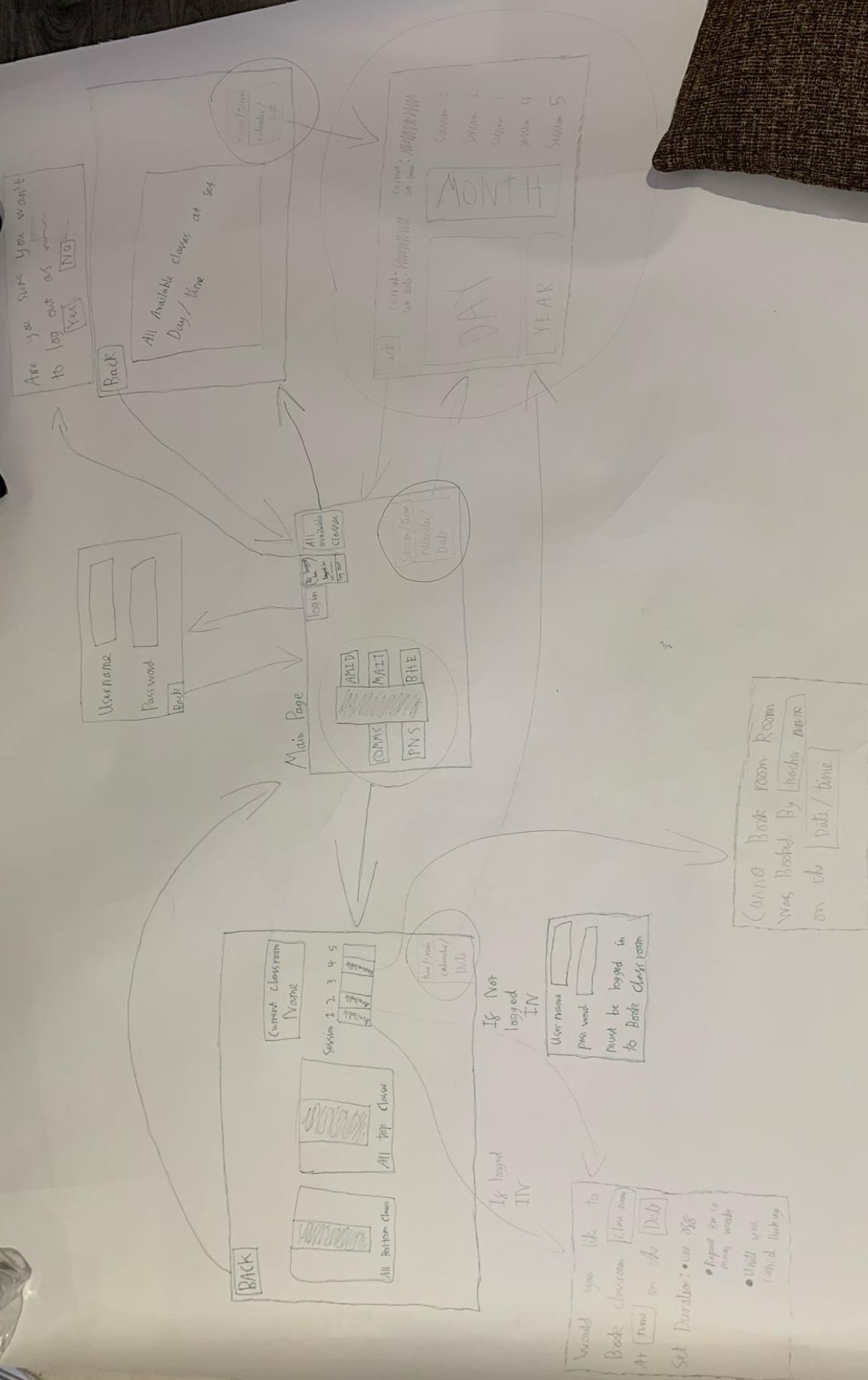
The bottom section, if row==24, is what counts how many widgets are being placed on the same column. Once too many have been put on the same column, we need to shift to a different column or else the widgets will go off the screen and the user will not be able to actually use the widgets! Once the row is 24, it sets back to 2 and adds 1 to the column so the new widgets will now be shifted to the next column. Each widget is given a command, because the widgets created from listoffreeclasses are all buttons. Each widget, however, have different arguments that will go into the function is selected. We will talk about the book_room function later. FLN and session_time are the arguments provided. Session_time is provided from the array sessiontime. That withholds 5 different values, the times of sessions in the school. While FLN is provided by the very value listoffreeclasses[x][y]. This is the name of the current classroom where a widget is being created for them.

RESULT

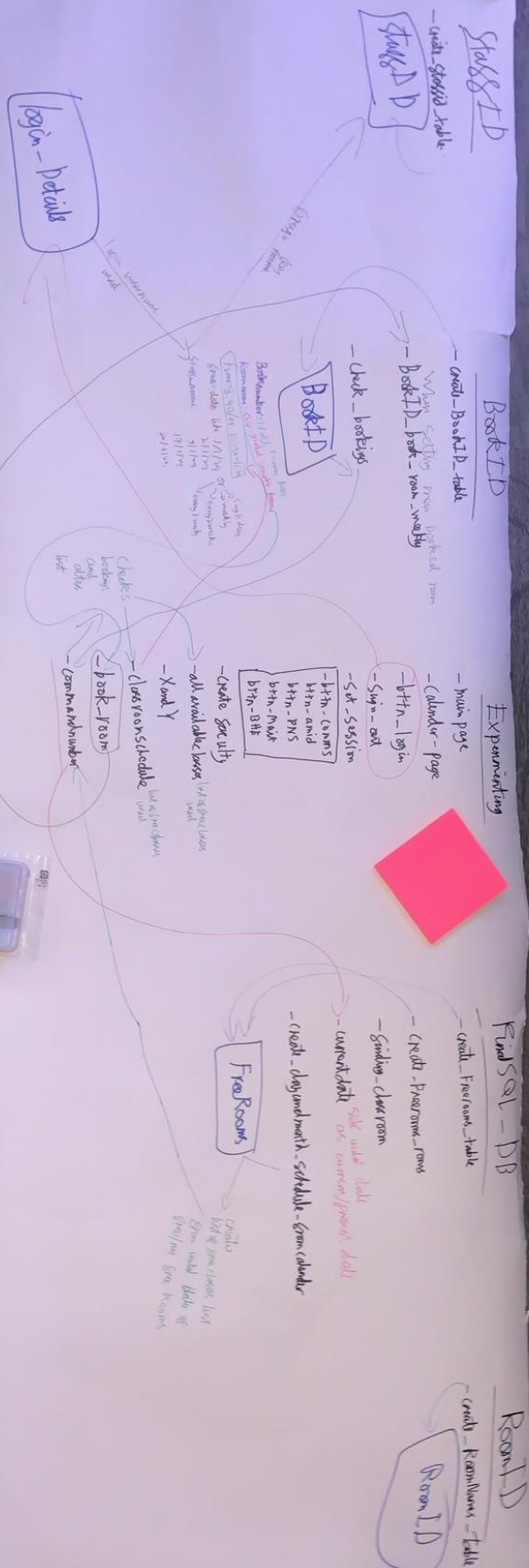
As you can see, the XandY widgets are seen, later when I am done with creating the UI I will make the foreground and background of those widgets to black so they will still be utilised but unseen.



Entire UI written down on A1 paper



All functions written down and shown how they connect



Not all the plans I wrote down will be in the final program. This is a sketch I made that I think what my program will look like at the end.

EXTERNAL DATABASES:

I have a total of four databases, BookID, FreeRooms, RoomID and Staff ID. Each hold different types of data and some use the data from the others.

BookID

When a user books a room, this is where the data is stored. Each record is a booking from any and all users. There are 5 fields, Booknumber integer, RoomName string, time string, freq string, staff name str, the primary key in this table is the BookNumber. With all of this data, they know who books, when they book, which rooms they want to book and how often.

```
def create_BookID_table():
    cursor.execute('''
CREATE TABLE BookID(
BookNumber int,
RoomName str,
time str,
freq str,
Staff_name str,
Primary Key(BookNumber)
```

BookNumber - The way the booknumber is created by counting the number of existing records then adding one in integer format. So, each record is unique and can be found specifically if

```
def BookID_book_room_weekly(Roomname,time,frequency,currentdate,Staff_name,value):
    if value == 1:
        Booknumber = currentbooknumber() + 1
        query = """INSERT INTO BookID(BookNumber,RoomName, time, freq, Staff_name) VALUES ('{B}', '{R}', '{T}', '{F}', '{S}');"""
        val = query.format(B=int(Booknumber),R=Roomname,T=time,F=frequency,S=Staff_name)
        cursor.execute(val)
        cursor.commit()
```

need be. I have created a function to count the number of records and return the value in integer form.

RoomName - This holds the room names of the classrooms the user is booking such as 'ML3', 'PU7',... it is held in a list and is imported into the python file from the main file.

Time - In this field, the session time is defined, such as '8:40 - 9:40' or '14:00 - 15:00'.

Freq - The dates of when the classrooms selected are booked, I have created a mechanism so whenever the system is running it checks for outdated bookings and then gets rid of them. This removes redundant data, saving space and memory so the program can run smoother. Such as '23/04/21', '10/11/20'.

Staff name - When the user is logged in, they receive a constant variable that is used whenever they book a room, this can allow for an algorithm to list all bookings under the name of the currently logged in user.

```
def currentbooknumber():
    row = cursor.execute('SELECT BookNumber FROM BookID')
    row2 = len(row.fetchall())
    cursor.commit()
    return (row2)
```

This program is a separate python file from the main one. This one sorts out the Bookings made from logged in users. This can also be used later when a page is created to show all bookings made under the name of the logged in user. Here we create the database, I put in a function so we can reset the entire database since I will be experimenting with the database. With each field you need to state the data type that will be stored in

that column / field. Such as time will be a string. Since it will be something like 8:40-9:40. And BookNumber will be 1,2,3 which can be in integer form. And used as primary key do to each record having a different Booknumber.

```
def create_BookID_table():
    cursor.execute('''
CREATE TABLE BookID(
BookNumber int,
RoomName str,
time str,
freq str,
Staff_name str,
Primary Key(BookNumber)
Foreign Key(Staff_name) REFERENCES StaffID(Staff_name)
Foreign Key(RoomName) REFERENCES RoomID(RoomName);''')
    cursor.commit()
```



```

1 import sqlite3
2 from datetime import datetime
3 from datetime import timedelta
4
5
6 def create_BookID_table():
7     cursor.execute('''
8     CREATE TABLE BookID(
9     BookNumber int,
10    RoomName str,
11    time str,
12    freq str,
13    Staff_name str,
14    Primary Key(BookNumber)
15    Foreign Key(Staff_name) REFERENCES StaffID(Staff_name)
16    Foreign Key(RoomName) REFERENCES RoomID(RoomName);''')
17    cursor.commit()
18
19 def check_for_outdates():
20     d1 = datetime.now()
21     d2 = d1.strftime("%d/%m/%y")
22     row2 = cursor.execute('SELECT BookNumber,freq FROM BookID')
23     row = row2.fetchall()
24     for i in range(len(row)):
25         temp = row[i][1]
26         temp2 = temp.strip("(").strip(")")
27         temp3 = []
28         for m in range(len(temp2.split(","))):
29             temp3.append(temp2.split(",")[m].strip(" ").strip("(").strip(")"))
30             if temp3[-1] <= d2:
31                 del temp3[-1]
32         number = row[i][0]
33         if len(temp3) == 0:
34             cursor.execute('DELETE FROM BookID WHERE BookNumber = ?', (number,))
35             cursor.execute('Update BookID set freq = ? where BookNumber = ?', (str(temp3), number))
36     row2 = cursor.execute('SELECT * FROM BookID').fetchall()
37     cursor.commit()
38
39
40 def BookID_book_room_weekly(Roomname,time,frequency,currentdate,Staff_name,value):
41     if value == 1:
42         BookNumber = currentbooknumber() + 1
43         query = """INSERT INTO BookID(BookNumber,RoomName, time, freq, Staff_name) VALUES ("{}","{}","{}","{}","{}");""".format(BookNumber,R=Roomname,T=time,F=frequency,S=Staff_name)
44         cursor.execute(val)
45         cursor.commit()
46     else:
47         Raxdate = datetime(2021,12,31)
48         row2 = cursor.execute('SELECT BookNumber FROM BookID')
49         row = row2.fetchall()
50         BookNumber = (len(row))+1
51         def weekly(adddate):
52             alldates = []
53             alldates.append(currentdate)
54             days = int(currentdate.split("/")[-1][0])
55             month = int(currentdate.split("/")[-1][1])
56             year= 2021
57             date1 = datetime(year, month, days)
58             date2 = date1 + adddate
59             while date2 <= Raxdate:
60                 newdate = (date2.strftime("%d-%m-%y"))
61                 month = (newdate.split("-")[1])
62                 days = (newdate.split("-")[0])
63                 newdate2 = (str(days)+"-"+str(month)+"-21")
64                 alldates.append(newdate2)
65                 date2 = date2 + adddate
66             query = """INSERT INTO BookID(BookNumber,RoomName, time, freq, Staff_name) VALUES ("{}","{}","{}","{}","{}");""".format(B=BookNumber,R=Roomname,T=time,F=allDates,S=Staff_name)
67             val = query.format(B=BookNumber,R=Roomname,T=time,F=allDates,S=Staff_name)
68             cursor.execute(val)
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111

```

Importing SQLite3, a module that holds the instruction set of SQL programs and codes.

Here we get the current time, ready to compare to the dates stores in the database BookID.

Here we get the frequency times along with their BookNumber and strip away the brackets and commas. So, we can compare them to the current date.

```

def check_for_outdates():
    d1 = datetime.now()
    d2 = d1.strftime("%d/%m/%y")
    row2 = cursor.execute('SELECT BookNumber,freq FROM BookID')
    row = row2.fetchall()
    for i in range(len(row)):
        temp = row[i][1]
        temp2 = temp.strip("(").strip(")")
        temp3 = []
        for m in range(len(temp2.split(","))):
            temp3.append(temp2.split(",")[m].strip(" ").strip("(").strip(")"))
        if temp3[-1] <= d2:
            del temp3[-1]
        number = row[i][0]
        if len(temp3) == 0:
            cursor.execute('DELETE FROM BookID WHERE BookNumber = ?', (number,))
            cursor.execute('Update BookID set freq = ? where BookNumber = ?', (str(temp3), number))
    row2 = cursor.execute('SELECT * FROM BookID').fetchall()
    cursor.commit()

```

We compare all the dates in the frequency field in each record to the current date, if that date is equal to or smaller than the current date. If it smaller than or equal to the current date it is removed from the database. This is for keeping the database up to date, no need for redundant data that will not serve anymore practical use. This keeps memory usage low, and the program can run more efficiently.

```

75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148

```

```

150 row3 = row2[i].replace("[", "").replace("]", "")
151 row4 = row3.replace("'", "").replace(" ", "")
152 if datetochk == row4:
153     Q = 0
154 return 0
155
156 def remove_booking(FUN, session_time, Staff_name, freqs):
157     row = cursor.execute('SELECT BookNumber, freq FROM BookID WHERE RoomName = ? AND time = ? AND Staff_name = ?', (FUN, session_time, Staff_name,)).fetchall()
158     cursor.commit()
159     freq2 = freqs.replace(" ", "")
160     freq = freq2.split(",")
161     for y in range(len(row)):
162         row2 = row[y][1].split(",")
163         m = 0
164         for x in range(len(row2)):
165             row3 = row2[x].replace("[", "").replace("]", "")
166             row4 = row3.replace("'", "").replace(" ", "")
167             if row4 != freq[x]:
168                 m = 1
169         if m == 0:
170             cursor.execute('DELETE FROM BookID WHERE BookNumber = ?', (row[y][0],))
171             cursor.commit()
172     cursor.commit()
173
174 def restart_BookID_table():
175     cursor.execute('DROP TABLE BookID')
176     cursor.commit()
177     create_BookID_table()
178     print("Reset required, BookID DB reset successful")
179
180 cursor = sqlite3.connect('BookID.db')
181 row = cursor.execute('SELECT * FROM BookID')
182 temp = len(row.fetchall())
183 cursor.commit()

```

```

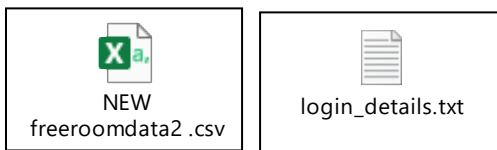
186 if temp > 0:
187     print("Connection to BookID successful, no restart required")
188     check_for_outdates()
189 else:
190     restart_BookID_table()
191

```

Here, the program checks if the database exists. If it, does It only run check for updates function, if it does not exist it uses the function at the start to create the database.

I began using text files to store data. I chose this because the data I received to use in my program was already in a text file (derived from edexcel). I thought keeping it in text file form and keeping everything in the same format would be easier than different data stored in different format. Then I created a program to use the data from freeroomdata2.txt to create the classrooms schedule. It was in vertical lines, and grouped in sessions. I began then storing login details on a text document and when logging in the program would validate the entered data to the data

in this text document



Here the program reads the text document

If the username is correct, it moves to check the password associated with the username

```

def validatelogin():
    global LOGGED_IN
    global username
    f = open('login_details.txt', 'r')
    data = f.readlines()
    d = 0
    for x in range(len(data)):
        t = 0
        data2 = data[x]
        data3 = data2.split(",")

        if username_enter.get() == data3[0]:
            t = t + 1
        if password_enter.get() + "\n" == data3[1]:
            t = t + 1
        if t == 2:
            list = roottwo.grid_slaves()
            for l in list:
                l.destroy()

            blackblock = Label(roottwo, width=1, height=1, highlightthickness=0)
            blackblock.configure(background='dark grey')
            blackblock.grid(row=0, column=0)

            logged_in = Label(roottwo, text="you are now logged in as " + str(data3[0]), width=50, height=1, highlightthickness=0)
            logged_in.configure(background='dark grey')
            logged_in.grid(row=0, column=1)

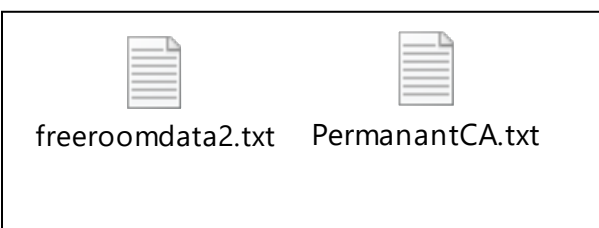
            roottwo.destroy()

            LOGGED_IN = 1
            username = str(data3[0])

            if chop == 1:
                list = root.grid_slaves()
                for l in list:
                    l.destroy()
                mainpage()

```

When it comes to booking, I had an idea of having the original plan of the classroom schedules stored on PermanantCA.txt and when anyone alters the schedules, it will be copied and the copy will be altered. Therefore, you will always have a copy of the original classroom's schedules.



If freeroomdata2.txt is identical to PermanantCA.txt the copy will be deleted and the program will go back to using PermanantCA.txt for the classrooms schedule.

I used a fourth text file named ALLFT. This would be used for all available classroom page. I didn't get far into programming this part because I realised text files wont do, I needed a more reliable tool to store data. Too many things went wrong when storing and extracting data from the text files, That is when I looked into SQL.

Structured Query Language. This is used to communicate with databases, and is known to be the standard language for rational database management systems. Firstly, I looked into replacing login_details.txt with a database. I ended up creating StaffID.



StaffID.db

```
import sqlite3
login_details = [{"Martin Corbishley", "MCORB", "cake", "martin.corbishley@corbybusinessacademy.org", "Computer Science"}, {"Heather thompson", "HTHOM", "cake", "heather.thompson@corbybusinessacademy.org", "Mathematics"}]
```

```
def create_StaffID_table():
    cursor.execute("""
CREATE TABLE StaffID(
Staff_name str,
username str,
password str,
email str,
subject str,
primary Key(Staff_name))""")
```

Creates StaffID database, instead of just a username and password, it stores the staff names, username, password, email and subject of teaching

```
for x in range(len(login_details)):
    if login_details[x][0] == 'admin':
        query = ("INSERT INTO StaffID(username) VALUES('"+username+"')")
        query_command = query.format(username=login_details[x][0])
        cursor.execute(query_command)
    else:
        alter = ("INSERT INTO StaffID (Staff_name,username,password,email,subject) VALUES ('"+name+"','"+username+"','"+password+"','"+email+"','"+subject+"')")
        alter_command = alter.format(name = login_details[x][0], username=login_details[x][1],password=login_details[x][2],email=login_details[x][3],subject=login_details[x][4])
        cursor.execute(alter_command)
        cursor.commit()
```

```
def checklogindetails(entered_username,entered_password):
    row = cursor.execute("SELECT username,password,Staff_name FROM StaffID")
    temp = row.fetchall()
    details = []
    for x in range(len(temp)):
        if entered_username == temp[x][0]:
            if entered_password == temp[x][1]:
                details.append(temp[x][0])
                details.append(temp[x][2])
            return details
    cursor.commit()
```

Here, the program checks the details in the database, the main program calls in this function from an external python file.

```
def restart_StaffID_table():
    cursor.execute("DROP TABLE StaffID")
    cursor.commit()
    create_StaffID_table()
    print("Reset required, StaffID DB reset successful")
```

Like we saw earlier, another function to check if the database already exists or not.

```
cursor = sqlite3.connect('StaffID.db')

row = cursor.execute("SELECT * FROM StaffID")
temp = len(row.fetchall())
cursor.commit()

if temp == len(login_details):
    print("Connection to StaffID successful, no restart required")
else:
    restart_StaffID_table()
```

Test Number	Test Criteria	Expected Output	Actual Output	Success (green, amber, red)
1	User must be presented a utility to log in	A button that opens a page for the user to input login details	 login Button.mp4	
2	System must accept valid login details	Once MCORB for username and cake for password is acceptor	 accept login details.mp4	
3	System does not allow invalid login details	Any random data inputted in is denied, and user is notified	 invalid login details.mp4	
4	Once the user has logged in, they have an option to sign out	Once logged in, a sign out button is presented and can log user out	 singout button.mp4	
5	The user can choose what date they are presented	The user selects a date and the information that is being presented changes	 data changes to date.mp4	
6	When the user changes the date, the system presents the new date that has been selected	The date that is being presented to the user changes when a new date is selected	 data shown to user.mp4	
7	Classes are separated by faculties	The user can select any of the five faculties	 faculties.mp4	
8	A page shows all classes available on selected date, grouped by session times	User can navigate to page that shows all classes available on present date	 all classes on date.mp4	
9	Classroom daily schedule is shown when grouped by faculties	User can see all classrooms daily schedule	 classrom daily schedule.mp4	
10	Each unavailable classroom session states who it is booked under	User can see name or teacher who has booked the classroom at specific session time	 teacher who booked room.mp4	

11	User CANNOT book room if not booked in	User will try to book but be directed to login first	 must login to book.mp4	
12	User is directed to choosing booking dates if logged in	User will be taken to page to selected dates if logged in and selects to book room	 user logged in to book.mp4	
13	A page shows all bookings under logged in user	Once logged in, a button to show all bookings appears and shows user all bookings done under their name	 all booking page.mp4	
14	User can decide to book classroom using weekly patterns	The user can select to book the room every two weeks.	 book weekly pattern.mp4	
15	System updates classroom schedule	Classroom is booked and their schedule is updated	 update schedule.mp4	
16	User can pick any dates freely to book classroom	User books room under 5 random dates	 pick 5 dates.mp4	
17	Each class when unavailable shows list of students that are presently using that class	User will see a list of students when they select	 list of students.mp4	
18	Each classroom states what kind of classroom it is	ML3 will state it is a computer class.	ML3 provide no information on the resources it withholds.	

Evaluation

From the test plan, most of my success criteria was successful. I will talk about these ones first. First was the login system, there was a choice for the user to login on the main page. This is important because I have created a system where the user **MUST** be logged in to use the booking utility. With this, validity was the next objective. To make sure only specific login details would grant logged in **TRUE** state to the user. This was successful, by creating a database that held the required specific data that must be inputted. I created a database so in the future I can easily add more profiles for staff. If data is inputted and does not fit required data, the user is notified with a message **Incorrect login details**. If they successfully login, the main page updates and two buttons appear, a sign out button and all of your bookings button. The sign out button (clue in the name) allows the user to regress back into a logged in state of **FALSE**. The next aim is to allow the user to change the date that is being presented. So, they can observe the classrooms schedule and availability in the future. I achieved this by creating a 'calendar', where you can navigate through the months of the year and then select the weekdays in the months. I chose the 2021 calendar due me creating this program in 2021. With this, the current date selected must be shown clearly to the user because it is important information when it comes to booking. So, on all pages when booking occurs, the current date is clearly shown in text in the format, **'Current date: dd/mm/yyyy'**. With the classrooms there are two ways I have chosen to represent them to the user. One is grouped through faculties, and the other is grouped by session time. On the Mainpage I created a simplistic bird's eye view of the layout of my school. On this, the faculties are represented with buttons. If these buttons are selected it takes the user to an overview of the top and bottom floor of the faculty with the classrooms found in that faculty. These classrooms are represented by buttons, once selected you see that specific classrooms daily schedule on the current date that is selected. The other form of representing classrooms is by taking all classrooms on the selected date, and divide into groups by session time. For example, any classrooms available any 8:40 – 9:40 (session 1) will be put in one list. This is shown on its own page, a button on the main page will navigate to said page. This page is called 'All available Classes'. When booking, the user must be logged in. they must have gone through the logged in validation process if they wish to alter the classrooms available states. This has been achieved and if you try to book a room and not logged in, the program navigates to the logging in page. If the user is logged in, it allows the user to access the page to choosing the dates to book the selected classroom. I created three weekly patterns, every week, every two weeks or every three weeks. Or the user can freely choose all available classes and as many as they want. Then the BookID database holds this record of booking. This updates the states of the classroom straight away, so other separate users will see this alter in the classrooms new schedule.

The main system of a booking system and showing the schedule of all classrooms do work, however the less crucial objectives have not been met. Such as showing additional information of classrooms. One is informing the user what additional equipment the classroom has. Such as gas taps in science classrooms, computers in computer science and IT classrooms. Paintbrushes / pencils and paper in Art classrooms. This is not detrimental to the booking process however to a new staff member who is unfamiliar with the classrooms may cause annoyances. Another is when classrooms sessions are unavailable, it states who has booked that room at that date. I failed to add this to the final product due to time and also, I created the basic classrooms schedules randomly. Also, I have a limited number of profiles so this seemed like a possible waste of exercise in the end. A third one I missed out on was announcing a list of students who would attend that booked room. Along with their teacher (who would most probably be the same who booked the room). This information was lost due to a lack of data (names of students) to use along with insufficient time.

Overall, I believe my program meets a sufficient number of criteria to be used in a practical environment. If I would re-do this program, I would focus on decreasing global variables used in the program, plus adding the additional information of the classrooms the UI presents. Overall, I am content with my program and would be confident enough for my school to put it in use.